SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning for Environment and

Agent COURSE OUTCOME COVERED

CO1: Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)

Assessment Tool Weightage: 40%

Dr. G. A. Senthil

QUESTIONS: 10 Total Marks: 50 Annexure A

Descriptive Experiment Exam Duration: 2hrs ***Code of Conduct:***

I _K.Neeha(192425245) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Experiment 1: Installation of Reinforcement Learning Environment

Install all the required software and check that they work.

Step 1: Install Anaconda

Download and install Anaconda for your operating system.

Step 2: Open Anaconda Prompt

Create a new environment:

conda create -n rl_env python=3.10

conda activate rl_env

Step 3: Install libraries

pip install numpy

pip install matplotlib

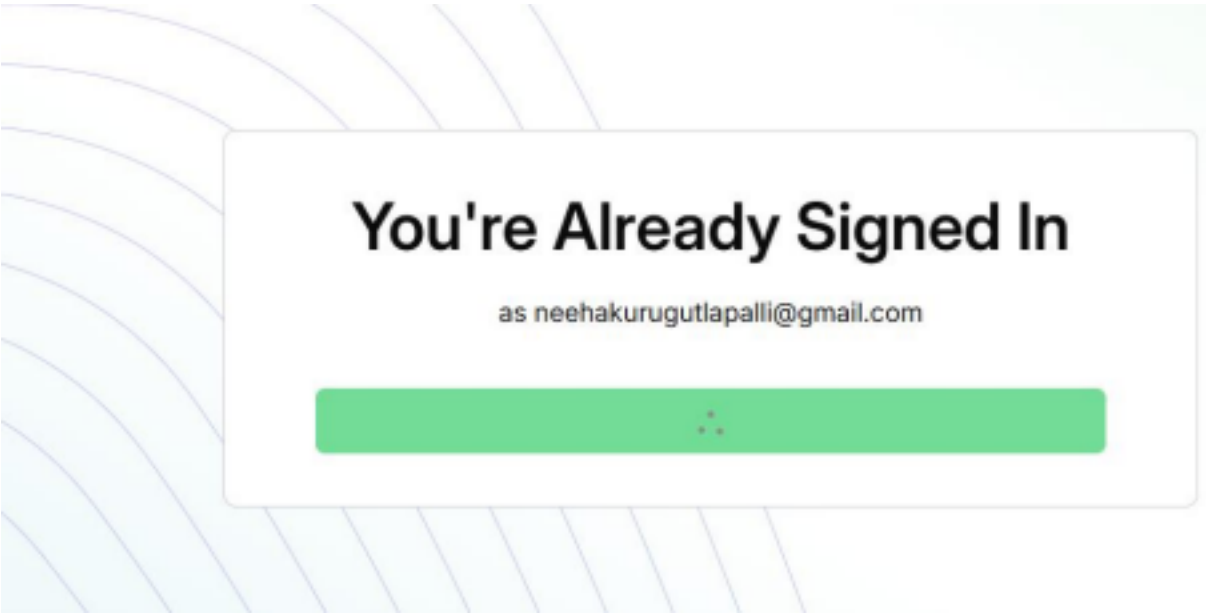
pip install gymnasium

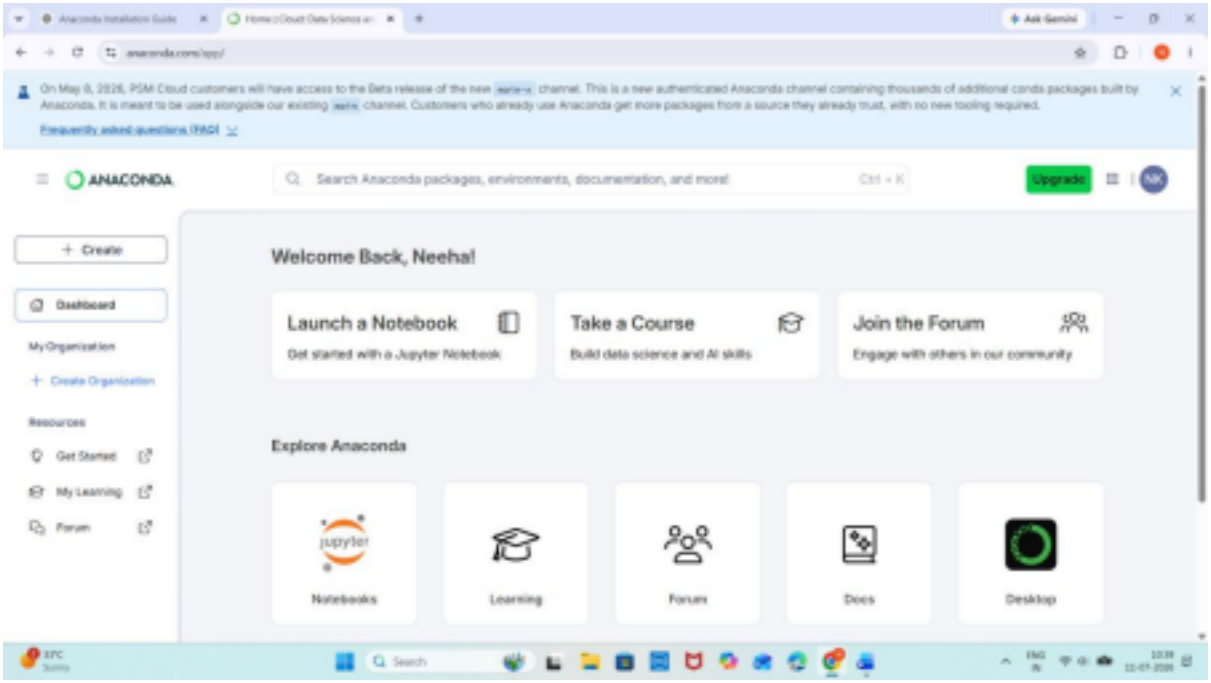
pip install tensorflow

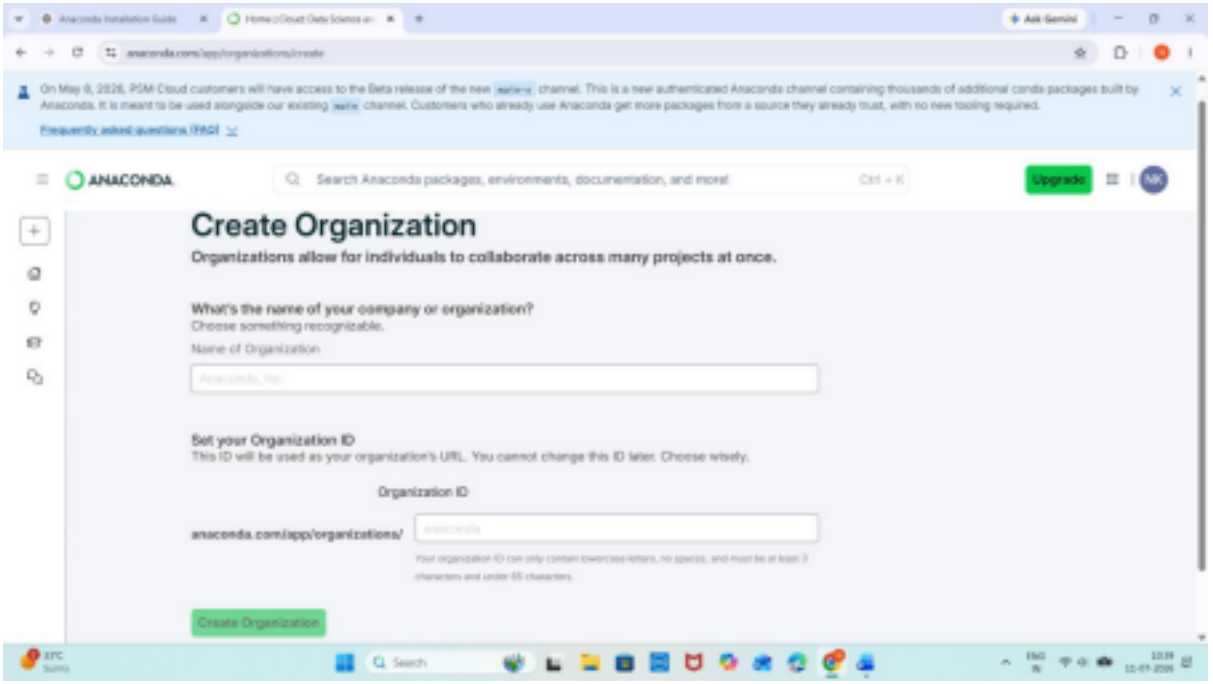
pip install keras

Step 4: Verify installation

```
import gymnasium
import tensorflow
import numpy
import matplotlib.pyplot as plt
print("Installation Successful")
```







Experiment 2: Familiarization with Gymnasium

Objective

Understand

- Environment
- State
- Action
- Reward

OUTPUT:

```
[6]
✓ 0s
for i in range(5):
    action = env.action_space.sample()
    state, reward, terminated, truncated, info = env.step(action)

    print(state, reward)

... 9 0
    8 0
    4 0
    8 0
    8 0
```

Experiment 3: Reinforcement Learning Framework

Learn this diagram

Agent

|

Action

|

Environment

|

Reward + Next State

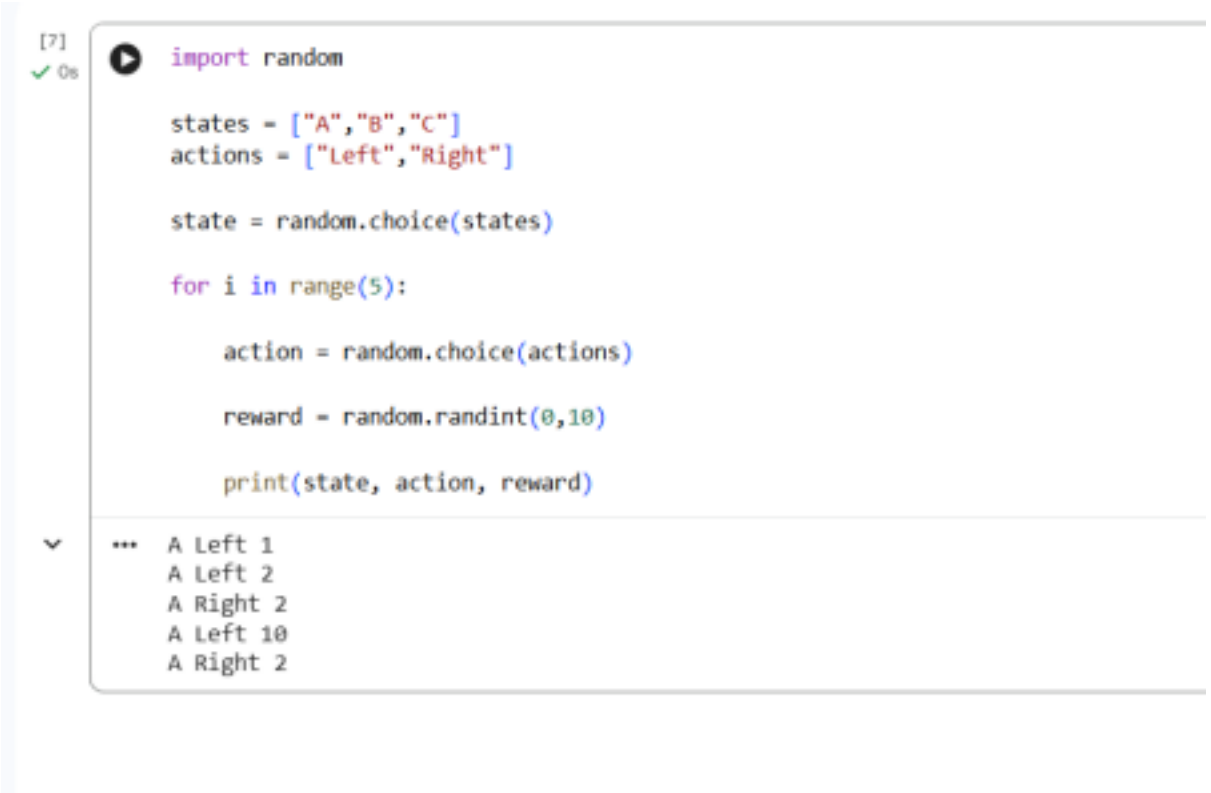
|

Agent

Program

```
import random


states = ["A","B","C"]
actions = ["Left","Right"]

state = random.choice(states)

for i in range(5):

action = random.choice(actions)

reward = random.randint(0,10)

print(state, action, reward)
```

OUTPUT:



Experiment 4: Markov Decision Process

Learn these terms

State

↓

Action

↓

Next State

↓

Reward

Example

Home

↓

Walk

↓

College

↓

Reward = 10

OUTPUT:

```
[8]
states = ["Home","College"]
transition = {
"Home":"College",
"College":"Home"
}
reward = {
"Home":0,
"College":10
}

current = "Home"

for i in range(4):

    print(current)

    current = transition[current]

    print("Reward =", reward[current])
```

```
Home
Reward = 10
College
Reward = 0
Home
Reward = 10
College
Reward = 0
```

Experiment 6: Exploration vs Exploitation

- Random
- Greedy
- ε-Greedy

Example

```
import random

values = [10,5,8]

epsilon = 0.2

for i in range(10):

 if random.random() < epsilon:
 action = random.randint(0,2)
 print("Explore", action)

 else:
 action = values.index(max(values))
 print("Exploit", action)
```

OUTPUT:

Experiment 8: Reward Visualization

```
import matplotlib.pyplot as plt
```

reward = [1,2,4,5,7,9,12]

plt.plot (reward)

plt.xlabel ("Episode")

plt .ylabel ("Reward")

plt.title ("Reward Graph")

plt. show()

OUTPUT:

Algorithm:

1. Initialize a list of rewards for multiple episodes.
2. Store reward values obtained in each episode.
3. Plot episodes on the X-axis and rewards on the Y-axis.
4. Analyze the trend of the graph.
5. Conclude the agent's learning performance.

import matplotlib.pyplot as plt

Sample rewards for 10 episodes

rewards = [10, 15, 20, 18, 25, 30, 35, 40, 45, 50]

episodes = list(range(1, len(rewards) + 1))
plt.plot(episodes, rewards, marker='o')

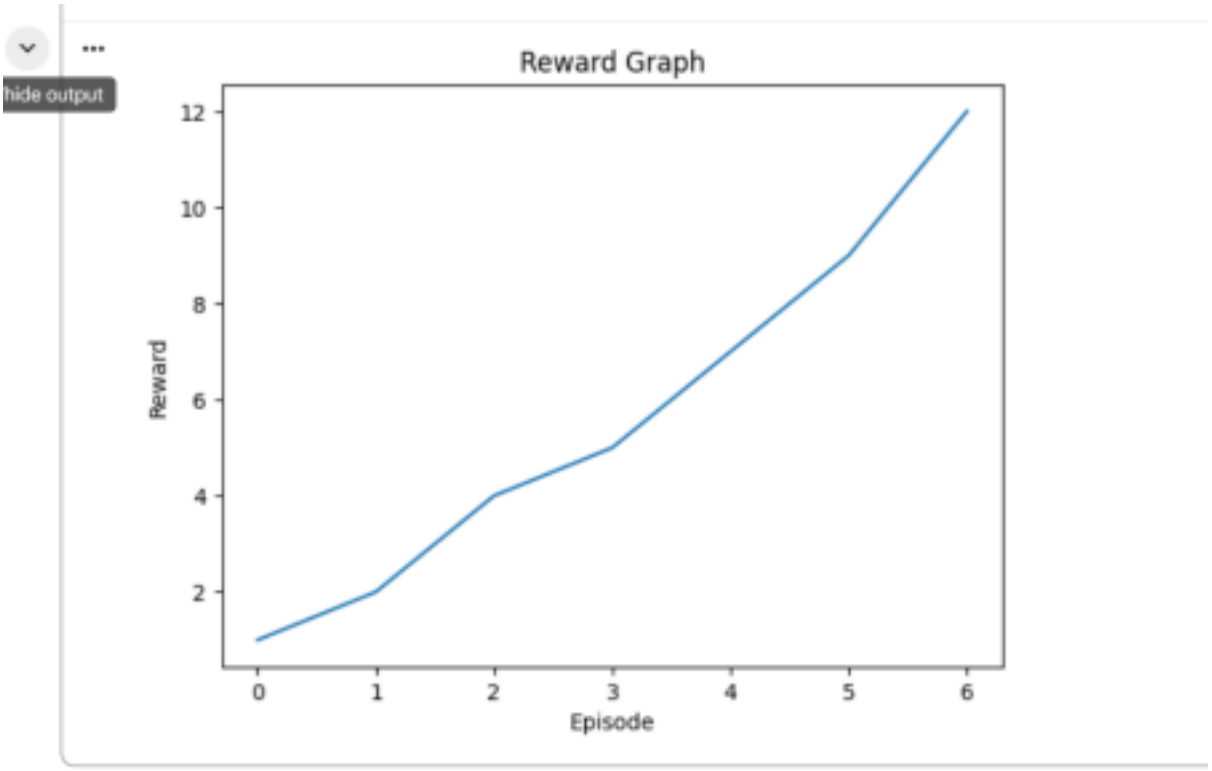
plt.title("Reward Visualization")

plt.xlabel("Episodes")

plt.ylabel("Rewards")

plt.grid(True)

plt.show()



Experiment 9: TensorFlow Neural Network

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
model = Sequential()
model.add(Dense(24,input_shape=(4,),activation='relu'))
model.add(Dense(24,activation='relu'))
model.add(Dense(2,activation='linear'))
model.summary()
```

OUTPUT:

```
[11]
✓ 0s
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential()

model.add(Dense(24,input_shape=(4,),activation='relu'))

model.add(Dense(24,activation='relu'))

model.add(Dense(2,activation='linear'))

model.summary()

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
dense_3 (Dense)	(None, 24)	120
dense_4 (Dense)	(None, 24)	600
dense_5 (Dense)	(None, 2)	50

```
Total params: 770 (3.01 KB)
Trainable params: 770 (3.01 KB)
Non-trainable params: 0 (0.00 B)
```

RUBRICS FOR CALCULATION

Criteria	Weightag e	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understandin g of Concepts	25%	Demonstrates thorough, accurate understandin g of concepts	Good understanding with minor gaps	Basic understanding with some misconception s	Poor understanding ; major misconceptions

Criteria	Weightag e	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Explanation	25%	Detailed, well explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis

Organizat 1		**Well structured answers with logical flow and coherence**	**Organized with minor issues in flow**	**Basic structure, lacks clarity**	**Poor organizatio n, incoherent answers**
Accuracy	**10%**	**Completel y accurate and covers all required points**	**Mostly accurate with minor omissions**	**Partially correct with missing points**	**Incorrect answers with major gaps**
Clarity	**5%**	**Clear, neat, profession al presentati on**	**Understand abl e with minor issues**	**Limited clarity, readability issues**	**Poorly written, difficult to understand**